

neUROn2++ Command Line Interface

May 24, 2009

1 Specify dataset

This menu choice includes all of the procedures related to specifying the dataset for the model. A dataset must be specified *before* any other step.

1.1 Number of input nodes

This menu choice allows you to set the number of input nodes. Its default is 1 input node, although that should *never* be used.

1.2 Number of output nodes

This menu choice allows you to set the number of output nodes. Its default is 1 output node.

1.3 Load raw dataset

This menu choice allows you to load a *raw* dataset from a file. The file must be in the same directory as the program. A raw dataset is one whose inputs and outputs have not yet been normalized. Generally, neural computational procedures require a dataset to be normalized, whereas statistical procedures do not. All datasets are of the format:

i_{11}	i_{12}	$\dots$	i_{1I}	o_{11}	o_{12}	$\dots$	o_{1O}
i_{21}	i_{22}	$\dots$	i_{2I}	o_{21}	o_{22}	$\dots$	o_{2O}
i_{31}	i_{32}	$\dots$	i_{3I}	o_{31}	o_{32}	$\dots$	o_{3O}
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$	$\dots$
i_{N1}	i_{N2}	$\dots$	i_{NI}	o_{N1}	o_{N2}	$\dots$	o_{NO}

Where i is an input to the model, and o is an output, and for N examples (“exemplars”) in the set, with I total inputs and O total outputs.

These columns may be separated in the file by spaces, commas, or any character which is non-numeric (1 through 9, ‘.’, e, E, - or + are not allowed as column separators.).

1.4 Convert a raw dataset into a training set

This menu choice will convert your raw dataset into a training set, by normalizing its inputs and outputs to those specified in the menu choice “specify variate bounds and types.” You will be asked if you want to change the number of thresholds used to calculate ROC area.

You will also be asked if you want to save your training set into a file. **This is generally recommended**, as you may want to load the training set at some point in the future to resume training. If you do save your training set, you will be asked for the filename to save its scaling factors. Scaling factors are used in the equation:

$$x_{norm} = lbound + S(x - min) \quad (1)$$

where x_{norm} is the normalized result, $lbound$ is the lower limit of the normalized x value (1 value for all), S is the scaling factor, and min is the minimum value of each x in the dataset. In this way, should the model be used in the future, you will be able to compute the normalized x_{norm} value from the x value that the user supplies.

1.5 Randomize a raw dataset into training and test sets

This menu choice will randomly divide your raw dataset into training and test sets. As it will attempt to keep the fraction of outcomes equal in both sets the same, the number of output nodes *must be* 1, and the output *must be* discrete (e.g. held to 0 or 1). You specify if an output is discrete in the menu choice “specify variate bounds and types.”

After the dataset has been randomized, you will be asked if you want to change the number of thresholds used to calculate ROC area, and if you want to save the newly created training and test sets to files.

As with converting a raw dataset into a training set, you will be asked if you want to save your training set and scaling factors into a file (see [equation 1](#)). You will also be asked if you want to save your test set into a file. Again, **this is generally recommended**.

You will be given the following choices to specify the number of exemplars to be placed in the test set, with the remainder to be placed in the training set (“ $N1/N2$ ” cross-validation):

1.5.1 As a whole number

This choice allows you to specify the number of exemplars in the test set as a whole number.

1.5.2 As a fraction with a numerator and denominator

This choice allows you to specify the number of exemplars in the test set as a fraction of the number of exemplars in the raw dataset. You will be asked for the numerator and the denominator of that fraction.

1.5.3 As a decimal fraction

This choice allows you to specify the number of exemplars in the test set as a fraction of the number of exemplars in the raw dataset. You will be asked for the decimal notation of that fraction (e.g. 0.25 would place $\frac{1}{4}$ of the exemplars from the raw dataset into the test set, and the remainder into the training set.)

1.6 Load training set

This menu choice allows you to load a training set from a file. The file must be in the same directory as the program. After the training set has been loaded, you will be asked if you want to change the number of thresholds used to calculate ROC area.

1.7 Save training set

This menu choice allows you to save your training set to a file, which will be saved in the same directory as the program. You will also be asked for the filename to save its scaling factors. See [equation 1](#) for an explanation of scaling factors.

1.8 Save scaling factors

This menu choice allows you to save the scaling factors of the training set, so that should the model be used in the future, you will be able to compute the normalized x_{norm} values from the x values that the user supplies. See [equation 1](#) for an explanation of scaling factors.

1.9 Load test set

This menu choice allows you to load a test set from a file. The file must be in the same directory as the program. After the test set has been loaded, you will be asked if you want to change the number of thresholds used to calculate ROC area.

1.10 Save test set

This menu choice allows you to save your test set to a file, which will be saved in the same directory as the program.

1.11 Log dataset operations to history file

Set this option to “yes” if you want the operations you perform in the “Specify dataset” menu to be logged to the history file **neuron.log**. This file will appear in the same directory as the program.

1.12 Specify variate bounds and types

In the following menu choices, variates are normalized according to the equation:

$$x_{norm} = lbound + [(\frac{x - min}{max - min}) \times (hbound - lbound)] \quad (2)$$

Where *lbound* is the lower boundary of a normalized value, *hbound* is the upper boundary, *min* is the minimum value of a variable *in the training set*, *max* is the maximum value of a variable *in the training set*, *x* is the variable's value and the normalized value of *x* is *x_{norm}*.

1.12.1 Output is discrete

Set this to “yes” if your outputs will always be either 0 or 1.

1.12.2 Threshold for discrete output

Set this to the number over which model outputs will be defined as 1, and under which model outputs will be defined as 0 for discrete output. For example, if the threshold is 0.5, a numeric model output of 0.6 will be transformed to 1, and a numeric model output of 0.3 will be transformed to 0. The default is 0.5.

1.12.3 Lower limit of input variates

Choose this menu item to set *lbound* for inputs in equation 2. The default is -0.9.

1.12.4 Upper limit of input variates

Choose this menu item to set *hbound* for inputs in equation 2. The default is +0.9.

1.12.5 Lower limit of output variates

Choose this menu item to set *lbound* for outputs in equation 2. The default is +0.1 if the output is not discrete (and obviously 0 if it is).

1.12.6 Upper limit of output variates

Choose this menu item to set *hbound* for outputs in equation 2. The default is +0.9 if the output is not discrete (and obviously 1 if it is).

1.13 Number of thresholds to calculate ROC area

Choose this menu item to set the number of thresholds over which ROC area is calculated. The output *must be* discrete, and the number of outputs *must be* 1 for ROC area to be calculated.

2 Specify model

Choose this menu item to specify a *neural computational* model. A dataset must be specified *before* this choice, and this menu choice must obviously be specified before “Use model”. Discriminant function analysis *does not* require this menu choice, and in fact, it is useless for discriminant function analysis.

2.1 Bias nodes

Choose this menu item to specify whether or not bias nodes will be included in your neural computational model. In general, for a real-world model, you should *always* include bias nodes. The default is “yes”.

2.2 Number of hidden layers / nodes on each layer

Choose this option to specify the number of hidden layers and number of nodes on each layer. The default is 1 layer with 1 node, although neural computational models will generally have > 1 hidden node. Most models will, however, only have 1 hidden layer.

2.3 Output error function

Choose this option to specify the output error function. “LMS” is least-mean-squared:

$$E = \frac{1}{2} \sum_{i=1}^m (y_i - o_i)^2 \quad (3)$$

where E is the error, m the number of outputs, y is the known value and o the model’s output.

You may also choose the cross-entropy error function “X-entropy”:

$$E = \sum_{i=1}^m -y \log(o) - (1 - y) \log(1 - o) \quad (4)$$

For the cross-entropy error function, the output *must be* discrete. Note that for sigmoidal output nodes, where for output $\vec{o}$,

$$\vec{o} = \frac{1}{1 + e^{-\vec{x}}}$$

if any element of $\vec{x}$ is very large or very small, the program will round to 0 or 1, and the log terms in the cross-entropy error function will cause an error. In such cases, an approximation is made (see the document **manifest.pdf** for a detailed explanation of that approximation), and the following error will be generated:

WARNING: Numerical out of bounds encountered when calculating error

2.4 Load network from a file

This menu choice allows you to load a neural computational model from a file. The file must be in the same directory as the program.

2.5 Binary logistic regression

This menu choice specifies binary logistic regression, which is mathematically equivalent to a single output node neural network with a bias, the sigmoidal transfer function, no hidden nodes, and the cross-entropy error function. *Always run a binary logistic regression model before starting to build a neural network.*

If your binary logistic model has acceptable goodness-of-fit, you don't need to build a neural network.

Note that at the end of a binary logistic run, β weights will be reported, along with their standard errors and p -values for the Wald test that the β weights are nonzero.¹ Note also that the indexing begins with input number 0, which is often referred to as the y -intercept. *In this case, β_0 is not the y -intercept.* It is the β weight of the first variate, indexed to 0. The y -intercept is clearly labeled as “ y -intercept” in the report table.

2.6 Log last operation to file

Set this option to “yes” (the default) if you want the last neural computational operation performed to be logged to the file **model.txt**. This file will be *overwritten* with each run, and will appear in the same directory as the program.

2.7 Log to history file

Set this option to “yes” (the default) if you want the neural computational operations that are performed logged to the file **neuron.log**. This file will be *appended* with each operation, and will appear in the same directory as the program.

3 Use model

Choose this menu item to use a specified *neural computational* model. A dataset must be specified *before* this choice, and the “Specify Model” menu must also have been entered before “Use model”.

3.1 Randomize weights

Choose this option to randomize the weights of the neural computational model. Prior to training the model, weights *must be* set, either by randomization or by

¹see Hosmer and Lemeshow, Applied Logistic Regression, 2nd edition, John Wiley & Sons, Inc., New York, 2000, pp 36–42

loading the model from a file in the “Specify model” menu. By default, weights are randomized between -0.5 and $+0.5$.

3.2 Set learning rate

Choose this option to set the learning rate η of the neural computational model. The default is $\eta = 1/N$ for on-line backpropagation (where N is the number of exemplars), and $\eta = 1$ for off-line (“batch” or “epoch”) training.

You may choose the learning rate to be “automatic” (see the Manifest documentation for a fuller explanation of automatic stepsize selection.) Briefly, automatic stepsize selection is designed to prevent oscillation around a local error minimum. Due to the increase in time required for this feature, it may be most optimal to train to near a local error minimum before setting automatic stepsize selection *on*.

3.3 Set stopping conditions

Choose this option to set conditions by which the neural computational model will stop training. These include:

3.3.1 Set maximum number of iterations

The model will stop training after the specified number of iterations. The default is 1000000, *which is arbitrary. Do not consider any neural computational model to be trained by stopping at any arbitrary threshold. You must inspect the model yourself to see if it has stopped training.*

3.3.2 Lower error limit

Use this choice to set the lower error limit that, if the error of the model decreases below, will result in cessation of training. For example, you could set this to the limit of your compiler. By default, no lower error limit is set.

3.3.3 Limit of change in error over 1 iteration

Use this choice to set the lower error limit that, if the change in error over 1 iteration is below, will result in cessation of training. Thus, in

$$\Delta E = E_{t+1} - E_t \tag{5}$$

Where E_t is the error after a training iteration, and E_{t+1} is the error after the next training iteration, if ΔE is less than the specified value, training stops. By default, this condition is not specified.

3.3.4 Stop if increase over error window width

Allows you to set a number of iterations that after which the error increases, training will stop. Thus, in

$$E_{n+\Delta} > E_n \quad (6)$$

Where n is the current iteration, E the error at an iteration, and Δ the error window width, training will stop if the inequality is satisfied. By default, this condition is not specified.

3.3.5 Maximum absolute gradient limit

All neural networks update their weights in the following form:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{g}_t \quad (7)$$

where $\mathbf{w}_t$ is the weight structure at time t , η is the learning rate, and $\mathbf{g}_t$ is the gradient. This menu choice allows you to stop training when the maximum absolute value of the elements in the gradient structure $\mathbf{g}_t$ decreases below the chosen limit. The default is 10^{-6} .

READ THIS!

The maximum absolute gradient limit should be the primary means by which you judge that training has ended, *not* by the error minimum.

For canonical backpropagation, using the maximum absolute gradient limit will significantly slow training (not by the number of iterations it takes to train, but by the amount of computational time consumed per iteration.) Thus, one strategy is to turn the maximum gradient choice off until you are near a local error minimum as judged loosely by the decrease in the error, then to turn on this menu choice.

Don't be surprised if the maximum absolute gradient fluctuates (increases and decreases.) Remember, there is more than one node and one layer in even the simplest neural network. Keep an eye on the overall trend of the maximum absolute gradient to insure it is decreasing.

3.4 Batch Epoch

Choose this option to set batch/epoch learning.

The *on-line* backpropagation learning algorithm is given by the formula:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{g}_k(\mathbf{w}_t) \quad (8)$$

where k is the exemplar index, $\mathbf{w}$ is the weight, t the iteration, $\mathbf{g}$ is the error gradient with respect to the weight, and η is the learning rate. By default, if on-line learning is chosen, $\eta = 1/N$, where N is the number of exemplars.

The *off-line*, also known as “batch” or “epoch” backpropagation learning algorithm is given by the formula:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{g}(\mathbf{w}_t) \quad (9)$$

where the *average gradient* $\mathbf{g}$ is defined by the formula:

$$\mathbf{g} = (1/N) \sum_{k=1}^N \mathbf{g}_k. \quad (10)$$

By default, if off-line learning is chosen, $\eta = 1$.

See the neUROn2++ Design Manifest for a more detailed description of on-line and off- line learning.

If batch/epoch is set *on*, off-line, “batch” or “epoch” learning is used. If batch/epoch is set *off*, on-line learning is used.

3.5 Weight Decay

The *error function*, E_k , for the k th exemplar is defined by the formula:

$$E_k(\mathbf{w}) = e(y^k, \mathbf{o}_k^{(m)}) + \frac{1}{2\sigma_w^2} \sum_{i=1}^m |\mathbf{w}|^2. \quad (11)$$

where y is the known or target outcome, and $\mathbf{o}$ is the model’s output. The second term on the right hand side of (11) is required to guarantee reliable statistical inference. It basically “stabilizes” the learning process by preventing the weights in the network from becoming too large or too small. σ_w is the standard deviation of the weights **chosen by the user**. Thus, a connection weight is expected to lie between $-1.96\sigma_w$ and $+1.96\sigma_w$ with probability equal to about 95%. Adjustment of this parameter can “change results” of the statistical analysis so σ_w is constrained to be exponents of 10 (e.g. 10, 100, 1000, ... 10^{12}). Once chosen, σ_w should not be changed.

A reasonable choice for σ_w might be $\sigma_w = 100$. This basically constrains the “learning process” to find connection strengths for the ANN such that the connection strength weight ranges between -200 and $+200$. The choice of σ_w should be reported in conjunction with all statistical analysis results. Note that as $\sigma_w \rightarrow \infty$, the effect of the summation term becomes negligible.

The learning rate equivalent of equation 11 is

$$\begin{aligned} \vec{w}_{t+1} &= \vec{w}_t - \eta \left. \frac{\partial E'}{\partial w} \right|_{w_t} \\ &= \vec{w}_t - \eta \left. \frac{\partial E}{\partial w} \right|_{w_t} - 2\eta\lambda\vec{w}_t \\ &= (1 - 2\eta\lambda)\vec{w}_t - \eta \left. \frac{\partial E}{\partial w} \right|_{w_t} \end{aligned}$$

where $(1 - 2\eta\lambda)$ is “weight decay”. λ may be set by either of 2 methods:

- The complexity of the population objective function $\gg$ sample objective function:

$$\lambda = \frac{1}{N2\sigma_w^2} \quad (12)$$

- The complexity of the population objective function $\simeq$ sample objective function:

$$\lambda = \frac{1}{2\sigma_w^2} \quad (13)$$

In general, as we don't know the complexity of the population objective function a priori, we choose equation 13 to calculate λ .

The following choices allow you to specify weight decay:

3.5.1 Don't change anything

Allows the uninformed user (which, assuming you've read this section, you are *not*,) to exit without changing the way in which weight decay is calculated.

3.5.2 `lambda = 1 / ( 2 * sigma^2 )`

Specifies that the weight decay term be calculated by

$$\lambda = \frac{1}{2\sigma_w^2}$$

and sets σ_w to be an exponent of 10 from 10^1 to 10^{12} .

3.5.3 `lambda = 1 / ( 2 * N * sigma^2 )`

Specifies that the weight decay term be calculated by

$$\lambda = \frac{1}{N2\sigma_w^2}$$

and sets σ_w to be an exponent of 10 from 10^1 to 10^{12} .

3.5.4 No weight decay

Turns off weight decay.

READ THIS!

Using too large of a weight decay term will degrade network modeling performance, but it is critical to use weight decay in order for statistical inference, for example, stepwise regression. Thus, setting a proper weight decay value is of utmost importance. Here is a good strategy to determine the proper weight decay value:

(Note that the default value for the weight decay term is $\frac{1}{2\sigma_w^2}$ with $\sigma_w = 100$. This is to prevent the uninformed user from making incorrect statistical inferences. However, you are no longer the uninformed user. In the first step, you will turn off weight decay.)

1. Turn weight decay off.
2. Experiment with different neural network architectures until you find one that is as near as possible a strict local minimum, and with as high ROC area as possible.
3. Save the neural network from Step 2.
4. Load the neural network from Step 2, and turn on weight decay. Use the choice for $\lambda = \frac{1}{2\sigma_w^2}$, and choose a low initial value for σ , for example 10^2 .
5. Train the neural network to as near a possible a strict local minimum (you'll be starting with the final weights from Step 2.)
6. Compare the ROC area to that from Step 2. If it is similar, go to step 8.
7. If the ROC area has substantially degraded, increase σ , load the neural network from Step 2, and go to Step 5.
8. Save the neural network, and record σ_w , as you will cite it when you publish your results. Perform stepwise regression with this value in the weight decay term, beginning with the final weights from the network you saved in this Step.

3.6 Print counter

This menu choice allows you to indicate whether printing will be linear (prints after a set number of iterations) or logarithmic. Logarithmic is recommended.

3.7 Train model

This menu choice initiates neural computational model training. You will choose the type of gradient descent. For an explanation of conjugate gradient descent and the Shanno algorithm, see Golden, pp. 217–218 and 221–222.² A brief explanation of these methods may also be found in the design manifest.

The menu choices are ordered by robustness of the algorithm, from less to more. Thus, conjugate gradient descent is less robust than the Shanno algorithm.

READ THIS!

Conjugate gradient descent and the Shanno algorithm generally are much faster at finding a local error minimum than canonical backpropagation *once one is near the local error minimum*. Thus, your training strategy should be to use canonical backpropagation until you are near a local error minimum (as judged by the maximum absolute gradient), and then apply either conjugated gradient descent or the Shanno algorithm. Empirically try each advanced algorithm, and choose the one to use based on your trials.

²Richard M. Golden, *Mathematical Methods for Neural Network Analysis and Design*, 1996, MIT Press, ISBN 0-262-07174-6, pp. 217–218, 221–222

3.7.1 Canonical backpropagation

This menu choice implements standard, plain vanilla backpropagation.

3.7.2 Conjugate gradient descent

This menu choice implements conjugate gradient descent. See the design manifest and Golden pp. 221–222.

3.7.3 Shanno algorithm

This menu choice implements the Shanno algorithm. See the design manifest and Golden pp. 217–218.

3.8 Save the network to a file

This menu choice allows you to save your trained neural computational model with its weights to a file. You may train the model repeatedly prior to saving it, if you so choose. If you have one output, and it is discrete, you will also be asked if you want to save the network's guesses to a file. These guesses will be saved as a 2 column file, with the first column being the known value, and the second column being the network's guess. You will be asked if you want to save guesses for the training set, and if it is loaded, the test set.

3.9 Stepwise regression

Stepwise regression allows you to determine which input variables are statistically significant to your trained neural computational model. For a more complete discussion, see the neUROn2++ Design Manifest. If you have not just finished training a neural network, you will be asked for the final error of the network model to regress.

3.9.1 Specify variables

A variable may be represented by more than 1 input node. For example, ethnicity may be represented by 4 nodes: African-American, Asian, Hispanic and Caucasian. Or a variable may be represented by 2 nodes: 1 in which the value is present or absent, and the other by the value of the variable itself. Variable structure may be input from the command line or loaded from file.

Input characters include:

- ; delimits variables
- , delimits nodes
- - specifies a range of nodes
- spaces are ignored

For example, “0-3,5; 4,8; 6,7” indicates that variable 0 is composed of input nodes 0,1,2,3 and 5, variable 1 is composed of input nodes 4 and 8, and variable 2 is composed of input nodes 6 and 7.

3.9.2 Stepwise reverse regression

Stepwise reverse regression proceeds by training the full network, and recording that network’s final error. Input variables are removed 1 at a time, the subnetworks so generated are trained to completion and their errors recorded, and the variable with the largest p -value (least significant) is then recorded with its p -value. Using the final error of the network which is deficient the one node with the largest p -value as a starting point, input variables are then removed 2 at a time, with each pair containing the 1 variable with the largest p -value and 1 of the remaining input variables. The pair of variables with the largest p -value (least significant) is then recorded with its p -value, and the process repeats until a chosen threshold p -value is reached.

Figure 1 demonstrates stepwise reverse regression.

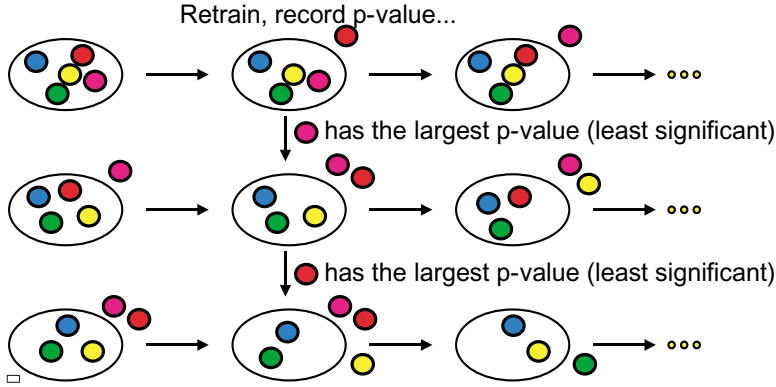


Figure 1: Stepwise reverse regression

3.9.3 Stepwise forward regression

Stepwise forward regression proceeds by training a baseline network with no input nodes, and recording that network’s final error. Input variables are added 1 at a time, the subnetworks so generated are trained to completion and their errors recorded, and the variable with the smallest p -value (most significant) is then recorded with its p -value. Using the final error of the network which is the one node with the smallest p -value as a starting point, input variables are then added 2 at a time, with each pair containing the 1 variable with the smallest p -value and 1 of the remaining input variables. The pair of variables with the smallest p -value (most significant) is then recorded with its p -value, and the process repeats until a chosen threshold p -value is reached.

Figure 2 demonstrates stepwise forward regression.

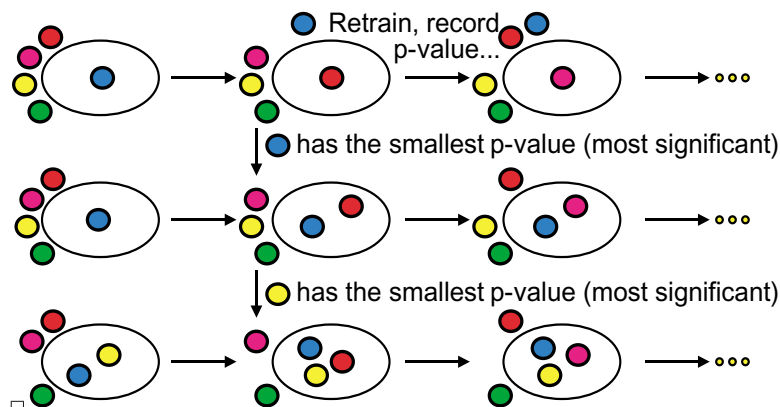


Figure 2: Stepwise forward regression

4 Discriminant function analysis

Choose this menu item to use discriminant function analysis to model your dataset(s).

4.1 Linear discriminant function analysis

Choose this menu item to model your dataset(s) with *LDFA*, linear discriminant function analysis.

4.2 Quadratic discriminant function analysis

Choose this menu item to model your dataset(s) with *QDFA*, quadratic discriminant function analysis.

4.3 Log last operation to file

Set this option to “yes” (the default) if you want the last statistical operation performed to be logged to the file **model.txt**. This file will be *overwritten* with each modeling procedure, and will appear in the same directory as the program.

4.4 Log to history file

Set this option to “yes” (the default) if you want the statistical operations that are performed logged to the file **neuron.log**. This file will be *appended* with each operation, and will appear in the same directory as the program.